

One More Pair, More Information Gained: Improved Attacks on LowMC with Full S-box Layers Using Two Plaintext/Ciphertext Pairs

Yimeng Sun^{1,2,3}, Jiamin Cui^{1,2,3,(✉)}, Shiyao Chen⁴, Meiqin Wang^{1,2,3},
Longzheng Cui^{1,2,3} and Chao Niu⁵

¹ School of Cyber Science and Technology, Shandong University, Qingdao, Shandong, China

² State Key Laboratory of Cryptography and Digital Economy Security, Shandong University,
Qingdao, 266237, Shandong, China

³ Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education,
Shandong University, Jinan, China.

⁴ Digital Trust Centre, Nanyang Technological University, Singapore

⁵ Ant Group, Beijing, China

sunyimeng@mail.sdu.edu.cn, cuijiamin@sdu.edu.cn, shiyao.chen@ntu.edu.sg,
mqwang@sdu.edu.cn, 202321179@mail.sdu.edu.cn, niuchao.niu@antgroup.com

Abstract. Motivated by LowMC cryptanalysis challenge, research in recent years focused more on attacking LowMC in PICNIC application setting, i.e. an attacker can see only a single plaintext/ciphertext pair. It can be noted that in the security proof of PICNIC, LowMC is required to be secure under two plaintexts, it is thus meaningful to investigate the security of LowMC in this direction. Pioneered by Liu, Isobe and Meier at Crypto 2021, they combined algebraic techniques with difference enumeration attack, which could attack all three 4-round LowMC instances adopting full S-Box layers in PICNIC with only two chosen plaintexts. However, the research on cryptanalysis of LowMC using two plaintext/ciphertext pairs is yet far from complete. Previous works using a single known plaintext are better than those with two plaintexts in terms of attack complexity or attacked rounds when considering a comparable success probability.

In this paper, to address such counter-intuitive gaps between existing attacks on LowMC with full S-box layers using a single and two plaintext/ciphertext pairs, we first develop an algebraic key-derived attack framework, where an algebraic property of the key-derived difference is utilized to build an equation system with lower algebraic degree. This directly contributes to less cost of solving equation system and naturally works under known-plaintext setting, which can be further enhanced with chosen-plaintext attack setting. We then present an improved difference enumeration attack framework. Instead of enumerating all possible differences in the second round, variables for part of S-boxes in the second and third rounds are introduced to derive cubic equations, which will lead to fewer variables for the last round. Finally, applying our new attack frameworks to LowMC, we propose 8-round attacks on LowMC for the very first time, which remain under known-plaintext setting. Moreover, we give the first attacks on three LowMC instances, i.e. 129-bit block size of 6 rounds and 129-/192-bit block size of 7 rounds, which cannot be obtained using previous attacking methods. Also, previous attacks on LowMC from 4 to 7 rounds could be improved for almost all three LowMC instances in this paper. All these results, we believe, could be a positive answer that given one more pair, more information indeed can be gained to improve attacks on LowMC when compared to those using only a single plaintext. As well as our newly proposed algebraic key-derived attack framework,

we hope, could provide more insights into the cryptanalysis of LowMC with low allowable data complexity.

Keywords: LowMC · PICNIC · linearization · algebraic attack · key recovery · polynomial method

1 Introduction

The LowMC family of block ciphers was originally proposed by Albrecht et al. [ARS⁺15] at Eurocrypt 2015, which was designed specifically for use cases in multi-party computation (MPC) and fully homomorphic encryption (FHE). To minimize the number of multiplications, LowMC is designed with a partial substitution-permutation (P-SPN) network with flexible parameters. As such, at CCS 2017, LowMC has been adopted as the underlying symmetric-key primitive in the post-quantum signature scheme PICNIC [CDG⁺17] due to its low multiplicative complexity, which was one of the third-round candidates in the NIST Post-Quantum Cryptography competition¹.

In light of the novel design strategy and a wide range of deployments in advanced cryptographic applications, LowMC has attracted much attention from the research community, especially for its resistance against key recovery attacks using a single plaintext/ciphertext pair. This is because in PICNIC, LowMC is used as a block cipher E as follows: a plaintext/ciphertext pair $(pt, ct = E_K(pt))$ will be made open as the public key (verification key) of the signature and encryption key K is used as the secret key (signing key). The security of PICNIC is directly related to the difficulty of recovering K from a single pair (pt, ct) . Therefore, to gain a deeper understanding of the security in PICNIC setting, Rechberger et al. announced a cryptanalysis challenge² for LowMC in 2020.

Soon after the publication of LowMC, the interpolation attack [DLMW15] and higher-order differential attack [DEM16] directly made LowMC move to LowMCv2 although with high data complexity. To adapt to the attack setting of low allowable data complexity, at ToSC 2018, Rechberger et al. [RST18] developed *difference enumeration technique* to analyze the security of LowMCv2 with partial nonlinear layers using 3 or slightly more *chosen* plaintext/ciphertext pairs and promoted the further update of LowMCv2 to LowMCv3. Evolving with LowMC versions, PICNIC has also continuously developed and to the latest PICNIC3 [pic20].³ Different from PICNIC2 [pic18] where a partial S-box layer is adopted when instantiating LowMC, a full S-box layer is used for three LowMC instances in PICNIC3. This change shows that LowMC parameter adopting a full S-box layer has gradually been preferable in PICNIC signature scheme when considering both security and efficiency, and LowMC instances with full S-box layers are *our main focus* in this work.

Related Works. In recent years, motivated by the cryptanalysis challenge, the research on cryptanalysis of LowMC focuses more on key recovery attacks in PICNIC setting, i.e. the attacker can only see a single plaintext/ciphertext pair under *known-plaintext setting*, which will directly affect the security of the signature scheme. Following this attack setting, cryptanalysis of LowMC has been a productive line of research [BBDV20, BBVY21, Din21, LMSI22, SCW23, BCV24] since 2020. Banik et al. [BBDV20] first proposed guess-and-determine attack on 2-round LowMC with full S-box layers, which was later improved to 3 rounds by using 2-stage meet-in-the-middle (MITM) attack with gray-code based approach at Asiacrypt 2021 [BBVY21]. In a parallel work at Eurocrypt 2021, Dinur [Din21] developed a polynomial method, the so-called Dinur’s algorithm, to attack all 4-round and two instances among three versions for 5-round LowMC instances with full S-box layers.

¹<https://csrc.nist.gov/projects/post-quantum-cryptography>

²<https://lowmcchallenge.github.io>

³For simplicity, we will use LowMC instead of LowMCv3 and PICNIC instead of PICNIC3 in the rest of the paper.

Later, by exploring different guess strategies to achieve better time-memory trade-offs, Liu et al. [LMSI22] significantly improved the memory complexity for 3-/4-round LowMC and Sun et al. [SCW23] reduced the huge memory cost for 5-/6-round LowMC. Recently, Banik et al. [BCV24] enriched Dinur’s algorithm with a specific linearization strategy, which yields the first attack on 7-round LowMC instance of 255-bit block size.

The optimal security reduction for PICNIC [CDG⁺17] requires the scheme to permit exactly two queries. To be fair, this setting of exactly *two* (pt, ct) pairs as special and relevant as impact on signature schemes can not be ruled out. On the other hand, the impact of an attack using 3 or more (pt, ct) pairs can be ruled out. Pioneered by Liu, Isobe and Meier [LIM21] at Crypto 2021, they reduced the differential enumeration attack to a problem of solving linear equations. Notably, with only *two chosen plaintexts*, they could attack all three recommended parameters of 4-round LowMC adopting full S-Box layers.

Motivations and Gaps. As one may expect, for an attacker given *one more* plaintext/ciphertext pair and even *strengthened* to be under chosen-plaintext setting, intuitively, *he should have achieved lower complexity for the same LowMC instance, and attacked more rounds or more instances of LowMC that cannot be reached before*. However, after revisiting previous attacks on LowMC instances with full S-box layers using a single plaintext and two chosen plaintexts, it has the following *counter-intuitive* observations:

1. For all three 4-round LowMC instances, previous attacks using only a single plaintext [Din21, LMSI22, BCV24] are even better than those with two chosen plaintexts [LIM21] in terms of the complexity and success probability.
2. There are dedicated attacks on 5-/6-/7-round LowMC instances using one plaintext/ciphertext pair. In contrast, with two available plaintexts, there is no related cryptanalysis yet as far as we know.

The attack with two chosen plaintexts is based on difference enumeration technique [RST18, LIM21], and we now dip into the potential limitation that leads to the phenomenon above. The difference enumeration attack consists of two steps: (i) the first step is to recover the compact differential trails, which is called the difference enumeration phase; (ii) the second step is to derive the secret key from the recovered differential trail in the first step, which is called the key-recovery phase. However, in the difference enumeration phase, the number of recovered differential trails will grow exponentially when increasing the number of attacked rounds for LowMC, i.e. exceeding the generic attack complexity when attacking more than 4 rounds. This will lead to a relatively high time complexity compared with previous attacks using only a single plaintext [Din21, LMSI22, BCV24], not to mention penetrating more rounds. Besides, the current attack method using one single plaintext cannot be trivially extended to make full use of two available plaintexts, i.e. yet to explore how to effectively gain more information from two plaintexts to improve complexity further.

With the gaps and challenges identified above, if an attacker achieve one more plaintext for LowMC instances with full S-box layers, i.e. using two plaintext/ciphertext pairs, we naturally ask the following two questions:

- Question 1. With two known-plaintexts, can an attacker gain more information to improve key recovery attacks compared with that using only one single plaintext?
- Question 2. When strengthened to be under chosen-plaintext setting, can such an attacker improve key recovery attacks further?

Our Contributions. In this work, to address the two questions raised above, we develop an algebraic key-derived attack framework and present an improved difference enumeration attack framework. Applying our new attack frameworks to LowMC with full S-box layers using two plaintext/ciphertext pairs, we give

- 8-round known-plaintext attacks on LowMC for the very first time, i.e. for LowMC instances with 192 and 255 bits block size;
- first attacks on three LowMC instances, i.e. $(n, k, m, R) = (129, 129, 43, 6)$, $(129, 129, 43, 7)$, $(192, 192, 64, 7)$;
- improved attacks for 4-/5-/6-/7-round of almost all three LowMC instances.

With these new results on LowMC, *Questions 1 and 2 can be answered positively*. The details of our results in this paper are summarized in Table 1.⁴ Besides, for a better and more visualized comparison between our results and previous related works, Fig. 1(a), Fig. 1(b) and Fig. 1(c) are provided. In the following, we introduce our two attack frameworks to further explain these results.

New Algebraic Key-Derived Attack Framework. Previous attacks [Din21, SCW23, BCV24] on the higher number of rounds of LowMC using only a single plaintext/ciphertext pair both follow a meet-in-the-middle method to build the equation system, then solve it to retrieve the key. The complexity of such an attack process is directly related to: 1) the number of variables; 2) the degree of the equation system. It can be noted that the follow-up dedicated works [SCW23, BCV24] introduced different guess strategies to achieve better trade-offs, i.e. reduce the number of variables or the algebraic degree, thus improving the attack complexity or attacking more rounds. Having these in mind, we now detail our new algebraic attack framework by following two parts.

Algebraic key-derived attack under known-plaintext setting. We first reveal an algebraic property of the key-derived difference to build an equation system with lower algebraic degree, which is derived from two known plaintext/ciphertext pairs and their differences. So, we purely explore in *reducing the algebraic degree of the built equation system* rather than other trade-offs to improve attacks on LowMC, which requires only under *known-plaintext attack setting* thanks to the proposed key-derived property. We then obtain the following new or better attacks compared to previous works [Din21, LMSI22, SCW23, BCV24] using only a single plaintext: *first 8-round attacks* on two LowMC instances with $(n, k, m, R) = (192, 192, 64, 8)$ and $(255, 255, 85, 8)$; first attacks on two LowMC instances with $(n, k, m, R) = (129, 129, 43, 6)$ and $(192, 192, 64, 7)$ *that cannot be attacked before*; better attacks on all three 4-round, 2 out of 3 of 6-round and one 7-round of LowMC instances.

Algebraic key-derived attack under chosen-plaintext setting. We further empower our algebraic key-derived attack with the ability of choosing inputs. Given that the equation system to solve for LowMC is built from a MITM method, one then should balance forward and backward rounds to reach an optimal choice of algebraic degree of the whole equation system. We observe that when the input difference is *chosen to be in a special form*, guessing some key bits in the first round can reduce the algebraic degree, which makes the attack more effective for attacks with an *odd number of rounds*. Thus, when enhancing to chosen-plaintext setting, some gaps for higher rounds of LowMC can be filled: we obtain the first attack on 7-round 129-bit LowMC instance that cannot be attacked in previous works and even our new attack under known-plaintext setting; we further improve attacks on LowMC instances with $(n, k, m, R) = (129, 129, 43, 5)$, $(192, 192, 64, 7)$ and $(255, 255, 85, 7)$ upon our new known-plaintext attack, which are also better than previous attacks [SCW23, BCV24, Din21] using only a single plaintext.

⁴To ensure a fair comparison, the complexity of the success probability lower than 0.5 is presented in Appendix C.

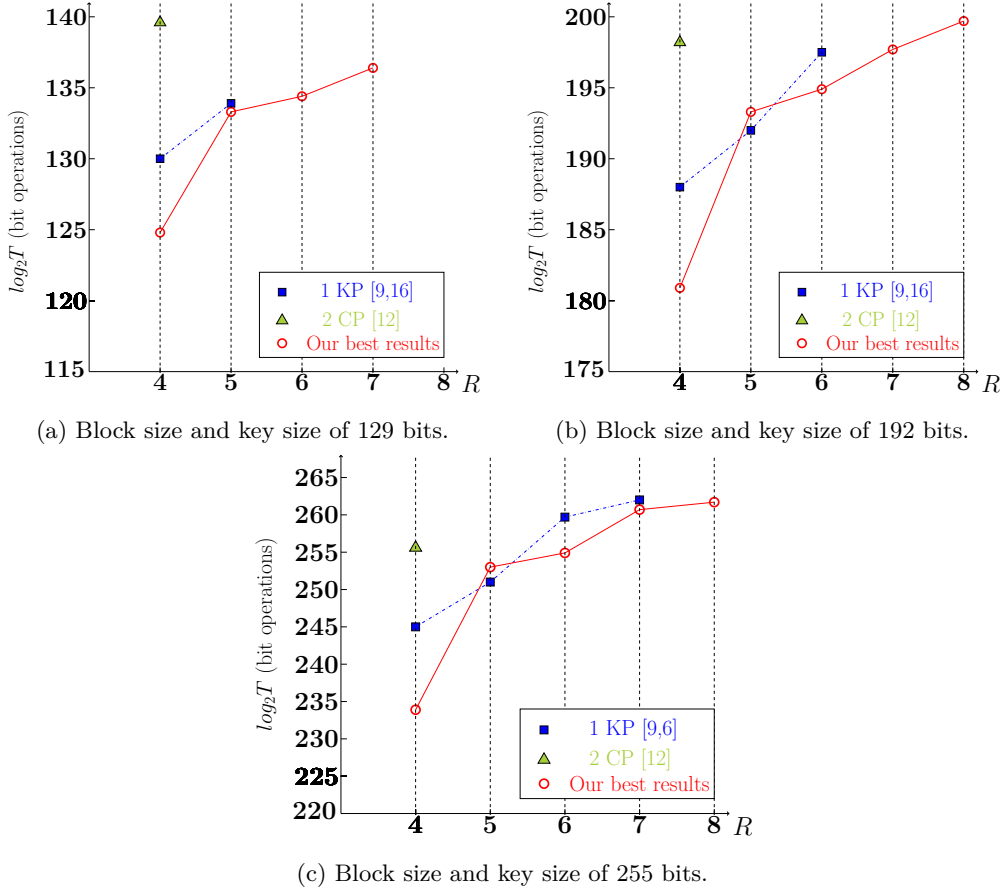


Figure 1: Comparisons between our results with two plaintext/ciphertext pairs and previous best attacks with one known plaintext or two chosen plaintexts.

Improved Difference Enumeration Attack Framework. We finally present an improved difference enumeration attack framework. The core idea is to avoid exhausting all possible differences in the second round, only part of differences in the second S-box layer will be enumerated. Then by introducing variables for the rest part of S-boxes in the second round and part of S-boxes in the third round, cubic equations can be set up, which will be conducive to derive a linear equation system with fewer variables to solve for the last round. Our improvements upon previous known-/chosen-plaintext attacks [Din21, LIM21, LMSI22, BCV24] and our known-plaintext attack, we believe, could jointly provide more insights into the cryptanalysis of LowMC with only two plaintext/ciphertext pairs from the aspect of difference enumeration attack.

Organization. The rest of the paper is organized as follows. In Section 2, some background knowledge is introduced. In Section 3, we propose a new algebraic key-derived attack framework for LowMC using two plaintext/ciphertext pairs. Then, an improved difference enumeration attack is presented in Section 4. The experimental results are provided in Section 5. Finally, we conclude the paper in Section 6.

Table 1: Summary of attacks on LowMC instances with full S-box layers.

R	Gray**	(n, k, m)	$d = 1$				$d = 2$				
			T	M	Pro. [†]	Ref.	$T^{\ddagger}$	M	Pro.	Ref.	KP/CP
4	134.8	(129,129,43)	130	113	0.69	[Din21]	139.6	-	0.62	[LIM21]	CP
			133.9	86.2	1.00	[BCV24]	124.8	34.3	0.63	Sec. 4	CP
			133.8	42.6	0.69	[LMSI22]	126.1	106.3	0.69	Sec. 3.1	KP
4	197.9	(192,192,64)	188	164	0.69	[Din21]	198.2	-	0.82	[LIM21]	CP
			192.9	123.8	0.64	[BCV24]	205.8	-	0.99	[LIM21]	CP
			195.0	61.2	0.69	[LMSI22]	180.9	52.2	0.83	Sec. 4	CP
							188.4	49.2	0.97	Sec. 4	CP
							182.1	154.5	0.69	Sec. 3.1	KP
4	261.0	(255,255,85)	245	218	0.69	[Din21]	255.6	-	0.848	[LIM21]	CP
			252.9	162.2	0.91	[BCV24]	265.6	-	0.986	[LIM21]	CP
			255.8	79.6	0.69	[LMSI22]	233.9	61.3	0.84	Sec. 4	CP
							244.8	63.7	0.99	Sec. 4	CP
							237.9	202.6	0.69	Sec. 3.1	KP
5	135.8	(129,129,43)	135.7	88.0	0.95	[BCV24]	134.4	122.9	0.69	Sec. 3.1	KP
			133.9	45.0	0.69	[SCW23]	133.3	45.0	0.69	Sec. 3.2	CP
5	198.9	(192,192,64)	192	173	0.69	[Din21]	194.9	180.7	0.69	Sec. 3.1	KP
			196.9	127.2	0.96	[BCV24]	193.3	174.0	0.69	Sec. 3.2	CP
			196.8	65.0	0.69	[SCW23]					
5	262.0	(255,255,85)	251	228	0.69	[Din21]	254.9	235.2	0.69	Sec. 3.1	KP
			256.9	166.0	0.82	[BCV24]	253.0	228.8	0.69	Sec. 3.2	CP
			258.9	84.6	0.69	[SCW23]					
6	135.8	(129,129,43)	-	-	-	-	134.4	122.9	0.69	Sec. 3.1	KP
6	198.9	(192,192,64)	198.0	128.5	0.52	[BCV24]	194.9	180.7	0.69	Sec. 3.1	KP
			197.5	65.6	0.69	[SCW23]					
6	262.0	(255,255,85)	259.7	168.5	0.52	[BCV24]	254.9	235.2	0.69	Sec. 3.1	KP
			260.5	86.0	0.69	[SCW23]					
7	136.8	(129,129,43)	-	-	-	-	136.4	39.8	0.69	Sec. 3.2	CP
7	199.9	(192,192,64)	-	-	-	-	199.8	183.1	0.69	Sec. 3.1	KP
							197.7	66.0	0.69	Sec. 3.2	CP
7	263.0	(255,255,85)	262.0	170.5	0.57	[BCV24]	261.7	246.1	0.69	Sec. 3.1	KP
							260.7	87.0	0.69	Sec. 3.2	CP
8	199.9	(192,192,64)	-	-	-	-	199.8	183.1	0.69	Sec. 3.1	KP
8	263.0	(255,255,85)	-	-	-	-	261.7	246.1	0.69	Sec. 3.1	KP

* T , M and Pro. represent the time, memory complexity (in $\log_2$) and success probability, respectively. d denotes the number of available plaintext/ciphertext pairs, T is given in the number of bit operations and M is given in bits. KP/CP means known or chosen plaintext setting.

** We take the complexity of Gray-code based exhaustive search technique in [BCC⁺10] as a baseline.

[†] We compare the results when the success probability is greater than 0.5. To ensure a fair comparison, the complexity of the success probability lower than 0.5 is presented in Appendix C. The success probability is not explicitly given in [Din21, SCW23]. For a more complete comparison, we calculate the corresponding success probability in Section 2.3.

[‡] We recalculate the complexity of [LIM21] in the number of bit-operations, e.g. one encryption of r -round LowMC takes about $2rn^2$ bit-operations.

2 Preliminaries

2.1 Notation

Due to the various parameters of LowMC [ARS⁺15], we use n , k , m , and R to denote block size in bits, key size in bits, the number of S-boxes per round, and the total number of rounds, respectively. Let $V = (a_0, a_1, \dots, a_{i-1})$ represents an i -bit vector and $V_0|V_1$

represents the composition of two vectors V_0 and V_1 . We denote $\binom{u}{\leq d} = \sum_{i=0}^d \binom{u}{i}$, where $\binom{u}{i}$ represents the binomial coefficient. The probability of the event X happens can be denoted as $Pr(X)$.

2.2 Description of LowMC

LowMC [ARS⁺15] is a family of SPN block ciphers proposed in Eurocrypt 2015. The parameter selection of LowMC is very flexible, and each user can choose the parameter independently and instantiate it. In this paper, we focus on the analysis of LowMC used in PICNIC3, so we only give the description of LowMC with full S-box layer.

Before the first round function, there is a whitening key (WK) denoted by K_0 . K_0 can be obtained by multiplying a random full rank matrix of size $n \times k$ with the master key K . Then, the round function iterates R times. The $(i+1)$ -th ($0 \leq i \leq R-1$) round function of LowMC consists of the following operations:

- SboxLayer (SB): A 3-bit S-box has Algebraic Normal Form (ANF):

$$S(x_0, x_1, x_2) = (z_0, z_1, z_2), \text{ where } z_0, z_1, z_2 \text{ satisfy}$$

$$z_0 = x_0 \oplus x_1 x_2, \quad z_1 = x_0 \oplus x_1 \oplus x_0 x_2, \quad z_2 = x_0 \oplus x_1 \oplus x_2 \oplus x_0 x_1.$$

The same 3-bit S-box is applied to the $n = 3m$ bits of the internal state in parallel.

- MatrixMul (M): The n -bit internal state is multiplied with an invertible matrix $M_i \in \mathbb{F}_2^{n \times n}$, which is chosen independently and uniformly at random from all invertible binary matrices.
- ConstantAddition (AC): An n -bit round constant C_i is XORed with the n -bit internal state. C_i is randomly generated.
- KeyAddition (AK): An n -bit round key K_{i+1} is XORed with the n -bit internal state, where $K_{i+1} = L_{i+1}(K)$ is derived from a master key K by multiplying with a random invertible $n \times n$ matrix.

Following the notation of [LIM21], the difference of the $(i+1)$ -th input state of SB is denoted by Δ_i and the difference of the corresponding output state is denoted by Δ_i^S . The difference of plaintexts is denoted by Δ_p , i.e. $\Delta_p = \Delta_0$. In our attacks, Δ_i and Δ_i^S will be represented as n -bit vectors.

$$\Delta_p \xrightarrow{WK} \Delta_0 \xrightarrow{SB} \Delta_0^S \xrightarrow{M} \xrightarrow{AC} \xrightarrow{AK} \Delta_1 \longrightarrow \cdots \longrightarrow \Delta_{R-1} \xrightarrow{SB} \Delta_{R-1}^S \xrightarrow{M} \xrightarrow{AC} \xrightarrow{AK} \Delta_R.$$

The **compact differential trail** is defined as below:

Definition 1. [LIM21] A differential trail $\Delta_0 \longrightarrow \Delta_1 \longrightarrow \cdots \longrightarrow \Delta_r$ is called a r -round compact differential trail when all (Δ_j, Δ_j^S) ($0 \leq j \leq r-1$) and Δ_r are known.

2.3 Methods for Solving Equation System of LowMC

Obviously, the problem of key-recovery attacks for LowMC with a given plaintext/ciphertext pair can always be reduced to solving a system of n equations in n key variables as

$$E(K) : f_0(K) = f_1(K) = \cdots = f_{n-1}(K) = 0, \quad (1)$$

where the algebraic degree of each f_i is bounded by $\deg(f_i) \leq \mathbf{d}$ and $K = (k_0, k_1, \dots, k_{n-1}) \in \mathbb{F}_2^n$. Split the n -bit key variables K into two subsets $K_1 = (k_0, k_1, \dots, k_{n-n_1-1})$ and

$K_2 = (k_{n-n_1}, k_{n-n_1+1}, \dots, k_{n-1})$ of size $n - n_1$ and n_1 . Then, $E(K)$ can be rewritten as $E(K_1, K_2)$. In this subsection, we revisit two of the current state-of-the-art methods of solving multivariate equation systems generated by LowMC. This will benefit our later attacks on LowMC.

Dinur’s Algorithm. Dinur’s algorithm is a general method for solving multivariate equation systems over $\mathbb{F}_2$.

Definition 2 (Isolated Solution [Din21]). A solution $\hat{K} = (\hat{K}_1, \hat{K}_2)$ to $E(K_1, K_2)$ is called isolated (with respect to the variable partition (K_1, K_2)) if for all $K'_2 \neq \hat{K}_2$, $(\hat{K}_1, K'_2)$ is not a solution to E .

Consider the equivalent representation $F(K_1, K_2) = \prod_{i=0}^{n-1} (f_i(K_1, K_2) \oplus 1)$. In order to recover $(\hat{K}_1, \hat{K}_2)$, we first take $\ell = n_1 + 1$ different equations at random from $E(K_1, K_2)$ and construct the new equation system $\tilde{E}(K_1, K_2)$, which can be expressed as

$$\tilde{E}(K_1, K_2) : \tilde{f}_0(K_1, K_2) = \tilde{f}_1(K_1, K_2) = \dots = \tilde{f}_{\ell-1}(K_1, K_2) = 0.$$

Similarly, we denote the corresponding equivalent representation as $\tilde{F}(K_1, K_2)$. It is obvious that a solution of E must be a solution of $\tilde{E}$. But a solution of $\tilde{E}$ is not always a solution of E . We choose to enumerate isolated solutions of $\tilde{E}$ instead of E and then verify them by E to take advantage of the low degree of $\tilde{F}(K_1, K_2)$. Then we can efficiently compute all $u_1 + 1$ partial sums to recover the isolated solutions. More details can be found in [Din21]. To reduce the cost of testing solutions, we perform N different choices for the equation system $\tilde{E}(x)$. The solution is retained and verified once it occurs more than once, so the success probability of Dinur’s algorithm can be estimated as

$$Pr_{[\text{Din21}]} = \sum_{i=2}^N \binom{N}{i} \cdot \left(\frac{1}{2}\right)^i \cdot \left(\frac{1}{2}\right)^{N-i}.$$

Therefore, $Pr_{[\text{Din21}]} \approx 0.69$ when $N = 4$ and $Pr_{[\text{Din21}]} \approx 1$ when $N = 16$.

The total time complexity can be evaluated in bit operations as

$$N \cdot (2 \cdot \mathbf{d} \cdot \log_2(n) \cdot 2^{n_1} \cdot \binom{n - n_1}{\leq d_{\tilde{F}} - n_1 + 1}) + N \cdot (n_1 + 1) \cdot (n - n_1) \cdot 2^{n - n_1}, \quad (2)$$

where $d_{\tilde{F}} = (n_1 + 1) \cdot \mathbf{d}$. We use $T_{[\text{Din21}]}(n, n_1, \mathbf{d}, N)$ instead of Equation (2) for convenience.

The memory complexity of Dinur’s algorithm is mainly dominated by storing the potential solutions, i.e. $M = N \cdot (n_1 + 1) \cdot 2^{n - n_1}$ bits. Using Dinur’s memory-efficient Möbius transform, the memory complexity of polynomial evaluation can be reduced to $M_{[\text{Din21}]} = 2 \cdot N \cdot (n_1 + 1) \cdot \binom{n - n_1}{\leq d_{\tilde{F}} - n_1 + 1}$ bits.

Improving Dinur’s Algorithm for LowMC Using Guess Strategy. In [SCW23], the authors improved the polynomial methods for LowMC equation systems based on the guess strategy in [LMSI22]. Assume an all-zero plaintext and consider the i -th S-box $S(k_{3i}, k_{3i+1}, k_{3i+2}) = (z_{3i}, z_{3i+1}, z_{3i+2})$ in the first round, $0 \leq i \leq m - 1$. Let $y_i = z_{3i}$. If we guess two input bits as $k_{3i+1} = g_{2i}$ and $k_{3i+2} = g_{2i+1}$, the 3-bit key variable takes value from the set

$$B^{g_{2i} g_{2i+1}} = \{(y_i \oplus g_{2i} g_{2i+1}, g_{2i}, g_{2i+1}) \mid y_i \in \{0, 1\}\},$$

where $g_{2i}, g_{2i+1} \in \{0, 1\}$. Then, after guessing $2m$ bits $G = (g_0, g_1, \dots, g_{2m-1}) \in \{0, 1\}^{2m}$, the first round can be linearized. We obtain n equations of degree $\mathbf{d}'$ for LowMC instance

over the partial space $K \in B^G = B^{g_0 g_1} \times \dots \times B^{g_{2m-2} g_{2m-1}}$ and $\mathbf{d}' < \mathbf{d}$. Denote the equation system by $E_G(K)$ as

$$E_G(K): f_{G,0}(K) = f_{G,1}(K) = \dots = f_{G,n-1}(K) = 0.$$

For any given $G = (g_0, g_1, \dots, g_{2m-1})$, there is a one-to-one correspondence between $K = (k_0, k_1, \dots, k_{3m-1}) \in B^G$ and $Y = (y_0, y_1, \dots, y_{m-1})$, i.e. $f_{G,i}(K) = \mathbf{f}_{G,i}(Y)$, $0 \leq i \leq n-1$. For any $0 \leq j \leq m-1$ we have $k_{3j} = y_j \oplus g_{2j} g_{2j+1}$, $k_{3j+1} = g_{2j}$, $k_{3j+2} = g_{2j+1}$, $y_j \in \{0, 1\}$. We call $\mathbf{f}_{G,i}$ the associated function of $f_{G,i}$ and Y the associated vector of K .

Consider the equivalent representation $F_G(K) = \prod_{i=0}^{n-1} (f_{G,i}(K) \oplus 1)$, then $F_G(K^*) = 1$ if and only if K^* is a solution of $E(K)$. Choose $n_1 = 3m_1$, where $1 \leq m_1 \leq m-1$. Let $\mathbf{F}_G$ denote the associated function of F_G . For a given G , to recover (K_1, K_2) , we first randomly take $l = n_1/3 + 1$ different equations from $\mathbf{E}_G(Y)$ and construct the new equation system $\tilde{\mathbf{E}}_G(Y)$ and the corresponding equivalent representation $\tilde{\mathbf{F}}_G(Y)$. Note that a solution of E_G must be a solution of $\tilde{E}_G$, but a solution of $\tilde{E}_G$ is not always a solution of E_G . We choose to enumerate solutions of $\tilde{E}_G$ and then verify them by E_G in order to benefit from the low degree of $\tilde{F}_G$. More details can be found in [SCW23]. To reduce the cost of testing solutions, we perform four different choices for the equation system $\tilde{E}_G(K)$. The solution is retained and verified once it occurs more than once, so the success probability can be estimated as

$$Pr_{[\text{SCW23}]} = \sum_{i=2}^4 \binom{4}{i} \cdot \left(\frac{1}{2}\right)^i \cdot \left(\frac{1}{2}\right)^{4-i} \approx 0.69.$$

The total time complexity can be evaluated in bit operations as

$$2^{2n/3} \cdot \left(\binom{N_u}{D} \cdot 2rn^2 \cdot 2^{n_1/3} + 4 \cdot (n_1/3 + 1) \cdot N_u \cdot \sum_{i=0}^{D-1} \binom{N_u - 1}{i} + D \cdot 2^{N_u} \right). \quad (3)$$

We use $T_{[\text{SCW23}]}(N_u, n_1, D)$ instead of Equation (3) for convenience. The memory complexity can be estimated as

$$M_{[\text{SCW23}]} = \binom{N_u}{D} \cdot n \cdot 2^{n_1/3} + 4 \cdot (n_1/3 + 1) \cdot 2^{N_u}$$

bits, where $N_u = (n - n_1)/3$ and $D = \mathbf{d}' \cdot l - n_1/3$.

Comparison of the Two Methods. The improved Dinur algorithm [SCW23] employs a guess strategy to partition the key space into smaller subspaces and solve the LowMC equation system, significantly reducing the memory complexity. However, the time complexity of [SCW23] usually increases by a small factor when the number of rounds is small. This mainly due to the high cost of linearization. When the number of rounds increases, the algebraic degree dominates the complexity and [SCW23] will do better.

2.4 The Difference Enumeration Attack for the Full S-Box Layer

The original difference enumeration attack proposed by Rechberger et al. in FSE 2018 [RST18] is not quite efficient for the vast memory consumption to store the pre-computed differences and the strong constraint on the number of recovered differential trails. So Liu et al. [LIM21] revisited the difference enumeration attack and developed the algebraic techniques at Crypto 2021. In this subsection, we briefly introduce the difference enumeration attack framework for full S-box layer in [LIM21]. This attack framework consists of the difference enumeration phase and the key-recovery phase.

Difference Enumeration Phase. In the difference enumeration phase, the algebraic technique to recover the compact differential trails is based on the following observation.

Observation 1. [LIM21] *For each non-zero input difference $(\Delta x_0, \Delta x_1, \Delta x_2)$, its valid output differences form an affine space of dimension 2. A similar property also applies to the inverse of the S-box.*

Denote the number of inactive S-boxes in the 2nd, 3rd and 4th rounds by q , j and t , respectively. The whole procedure is divided into the following four steps.

- Step 1. Find a suitable assignment for Δ_0^S such that the number of inactive S-boxes in the 2nd round can be maximized and there is only one active S-box in the first round.
- Step 2. Choose a value Δ_0 so that it can reach Δ_0^S . Encrypt two arbitrary plaintexts whose difference equals to Δ_0 , then collect their corresponding ciphertexts and obtain Δ_3^S .
- Step 3. Enumerate 4^{m-q} possible difference transitions from Δ_1 to Δ_2 . For each possible difference transition, move to Step 4.
- Step 4. For each obtained Δ_2 , we can introduce $(2n/3 - 2j)$ variables to represent Δ_2^S , and there will be $(n/3 + 2t)$ linear constraints at Δ_3^S according to Observation 1. Then we perform Gaussian elimination on them. For each solution of this equation system, a compact differential trail is obtained.

It can be depicted as

$$\Delta_0 \xleftarrow[\text{of inactive S-boxes}]{\text{Maximize the number}} \Delta_0^S \longrightarrow \Delta_1 \xrightarrow{\text{Enumeration}} \Delta_2 \longrightarrow \Delta_2^S \xrightarrow{\text{Solving equations}} \Delta_4.$$

Key-Recovery Phase. Once a compact differential trail is obtained, we can construct a linear equation system to recover the key based on Observation 2.

Observation 2. [LIM21] *For each valid non-zero difference transition $(\Delta x_0, \Delta x_1, \Delta x_2) \rightarrow (\Delta z_0, \Delta z_1, \Delta z_2)$, the inputs conforming to such a difference transition will form an affine space of dimension 1. In addition, (z_0, z_1, z_2) becomes linear in (x_0, x_1, x_2) , i.e. the S-box is freely linearized for a valid non-zero difference transition. A similar property also applies to the inverse of the S-box.*

For the 1st round and the 4th round, we only consider the active S-boxes. For each active S-box, we can get 2 linear equations about the plaintext/ciphertext and master key, i.e. $2 + 2(m - t)$ linear equations in total. For the 3rd round, we linearize all the j inactive S-boxes by guessing 2 output bits and there are $2m$ equations. In this way, all the input and output bits of S-boxes in the 2nd round can be expressed linearly in $(3t + n)$ variables. Since there are $(m - q)$ active S-boxes in the 2nd round, we can obtain $(m - q)$ equations. If $3t + n \leq 2 + 2(m - t) + 2m + 2(m - q)$, the equation system can be easily solved. If $3t + n > 2 + 2(m - t) + 2m + 2(m - q)$, we need to enumerate all the free variables and check them via plaintext/ciphertext pairs.

3 Algebraic Key-Derived Attack Framework for LowMC Using Two Plaintext/Ciphertext Pairs

In the previous works, the problem of the key recovery of LowMC under a single known plaintext/ciphertext pair can be reduced to the problem of solving linear/nonlinear equation system using some polynomial methods. The guess strategy is commonly used to reduce the number of variables or the algebraic degree of the equations and therefore achieves a better time-memory trade-off [BBVY21, SCW23]. In this section, we show how to

leverage the relationship between two plaintext/ciphertext pairs to reduce the algebraic degree of the equation system without extra guesses. In Section 3.1, we propose a new MITM algebraic attack framework under two known plaintext/ciphertext pairs. To further optimize the attack for odd number of rounds, we restrict the input difference into a special form and explore different active positions. Based on this, we propose two advanced MITM algebraic attack frameworks in Section 3.2.

3.1 MITM Algebraic Key-Derived Attack Framework

Let $Enc_K^{(R)}: \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an R -round LowMC block cipher defined by

$$Enc_K^{(R)}(X) = K_R \oplus F_{R-1}(\cdots F_1(K_1 \oplus F_0(K_0 \oplus X))),$$

where $K_{i+1} = L_{i+1}(K)$ is the round key of the $(i+1)$ -th round derived from a master key $K = (k_0, \dots, k_{n-1}) \in \mathbb{F}_2^n$ ($0 \leq i \leq R-1$) and $K_0 = L_0(K)$ is a whitening key. Let X denotes the plaintext. $F_i(\cdot) = C_i \oplus M_i \cdot SB(\cdot)$ denotes the $(i+1)$ -th round function. Obviously, $F_i(\cdot)$ is a quadratic function and the general expression for the output bit of the quadratic round function can be expressed by

$$F_i(U) = A \oplus \sum_{j=0}^{n-1} B_j u_j \oplus \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t} u_j u_t,$$

where $U = (u_0, \dots, u_{n-1})$ and $0 \leq i \leq R-1$. Denote the upper bound of the algebraic degree for K in $Enc_K^{(r)}(X)$ by $\delta_K(Enc_K^{(r)}(X))$. Obviously, we have $\delta_K(Enc_K^{(r)}(X)) = 2^r$. Consider another plaintext X' , we have

$$\Delta Enc_K^{(r)} = Enc_K^{(r)}(X) \oplus Enc_K^{(r)}(X').$$

The following theorem proves the upper bound of $\delta_K(\Delta Enc_K^{(r)})$.

Theorem 1. *Let $Enc_K^{(R)}: \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an R -round LowMC block cipher defined above. Given two plaintext X and X' , we have $\delta_K(\Delta Enc_K^{(r)}) = 2^r - 1$.*

A detailed proof of Theorem 1 can be found in the Appendix B. Using the above strategy, we can construct the equation system for arbitrary two plaintext/ciphertext pairs with a MITM method.

Even Number of Rounds. For an even number of rounds $r = 2\rho$, we consider the rounds 1 to ρ , the input difference of $(\rho+1)$ -th round Δ_ρ can be expressed by an equation system of degree $2^\rho - 1$ in key variables. Similarly, consider rounds 2ρ to $\rho+1$, the input difference of $(\rho+1)$ -th round Δ_ρ can also be expressed as an equation system of degree $2^\rho - 1$ in key variables. Then we can obtain n equations of degree $2^\rho - 1$ in n variables by simply equating the two sets of equations.

Odd Number of Rounds. For an odd number of rounds $r = 2\rho + 1$, we consider the rounds 1 to ρ , the input difference of $(\rho+1)$ -th round Δ_ρ can be expressed by an equation system of degree $2^\rho - 1$ in key variables. Similarly, consider rounds $2\rho + 1$ to $\rho+1$, the input difference of $(\rho+1)$ -th round Δ_ρ can also be expressed as an equation system of degree $2^{\rho+1} - 1$ in key variables. Then we can obtain n equations of degree $2^{\rho+1} - 1$ in n variables by simply equating the two sets of equations.

Complexity Evaluation. Once we have this equation system, we can use Dinur's algorithm to solve it. When $r = 2\rho$, we can get n equations of degree $2^\rho - 1$ for the n key variables, i.e. $\mathbf{d} = 2^\rho - 1$. When $r = 2\rho + 1$, we can get n equations of degree $2^{\rho+1} - 1$ for the n key variables, i.e. $\mathbf{d} = 2^{\rho+1} - 1$. Based on Equation (2) and choosing $N = 4$, the time complexity can be evaluated in bit operations as

$$T_{\text{KP}} = T_{[\text{Din21}]}(n, n_1, \mathbf{d}, 4).$$

The memory complexity can be evaluated in bits as

$$M_{\text{KP}} = 8 \cdot (n_1 + 1) \cdot \binom{n-n_1}{\leq \mathbf{d} \cdot (n_1+1) - n_1+1}.$$

3.2 Advanced MITM Algebraic Attack Framework

The meet-in-the-middle approach produces equations of degree $2^\rho - 1$ both the forward and backward directions for even rounds. But for odd rounds, the extra round causes a rapid increase in the time complexity due to the unbalance of the algebraic degree. In this subsection, we further optimize the attack framework for odd rounds when enhancing to chosen-plaintext setting. This improvement is based on a new observation of the S-box.

Specially, consider the LowMC S-box $S(x_0, x_1, x_2) = (z_0, z_1, z_2)$ as follows:

$$z_0 = x_0 \oplus x_1 x_2, \quad z_1 = x_0 \oplus x_1 \oplus x_0 x_2, \quad z_2 = x_0 \oplus x_1 \oplus x_2 \oplus x_0 x_1.$$

Let $(x_0, x_1, x_2) \oplus (x'_0, x'_1, x'_2) = (\Delta x_0, \Delta x_1, \Delta x_2)$. For $S(x_0 \oplus k_0, x_1 \oplus k_1, x_2 \oplus k_2) = (z_0, z_1, z_2)$ and $S(x'_0 \oplus k_1, x'_1 \oplus k_1, x'_2 \oplus k_2) = (z'_0, z'_1, z'_2)$, if we guess $(k_0, k_1) = (k_0^*, k_1^*)$, then we have

$$\begin{aligned} \Delta z_0 &= (\Delta x_0 \oplus \Delta x_1 \Delta x_2) \oplus \Delta x_1 (x_2 \oplus k_2) \oplus \Delta x_2 (x_1 \oplus k_1^*), \\ \Delta z_1 &= (\Delta x_0 \oplus \Delta x_1 \oplus \Delta x_0 \Delta x_2) \oplus \Delta x_0 (x_2 \oplus k_2) \oplus \Delta x_2 (x_0 \oplus k_0^*), \\ \Delta z_2 &= (\Delta x_0 \oplus \Delta x_1 \oplus \Delta x_2 \oplus \Delta x_0 \Delta x_1) \oplus \Delta x_0 (x_1 \oplus k_1^*) \oplus \Delta x_1 (x_0 \oplus k_0^*). \end{aligned}$$

Then if $(\Delta x_0, \Delta x_1, \Delta x_2) = (0, 0, 1)$, we have $(\Delta z_0, \Delta z_1, \Delta z_2) = (x_1 \oplus k_1^*, x_0 \oplus k_0^*, 1)$. Indeed, by choosing a special form of input difference, i.e. only one active bit, the output difference is determined by guessing 2 key bits. For example, if $(\Delta x_0, \Delta x_1, \Delta x_2) = (0, 1, 1)$ and $(k_0, k_1) = (k_0^*, k_1^*)$, we obtain $(\Delta z_0, \Delta z_1, \Delta z_2) = (1 \oplus x_2 \oplus k_2 \oplus x_1 \oplus k_1^*, 1 \oplus x_0 \oplus k_0^*, x_0 \oplus k_0^*)$. The observation can be summerized as follows.

Observation 3. *Let $S(x_0 \oplus k_0, x_1 \oplus k_1, x_2 \oplus k_2) = (z_0, z_1, z_2)$ and $S(x'_0 \oplus k_1, x'_1 \oplus k_1, x'_2 \oplus k_2) = (z'_0, z'_1, z'_2)$. If the input difference $(\Delta x_0, \Delta x_1, \Delta x_2)$ has only one active bit, then the output difference $(\Delta z_0, \Delta z_1, \Delta z_2)$ is determined by guessing two key bits which correspond to the 2 inactive bits of the S-box.*

By exploring the different positions of active bits, we propose the following two advanced MITM algebraic attack frameworks as depicted in Figure 2.

Activate Only One Bit. We consider the case that there is only one active S-box in the first round and this active S-box has only one active bit. Then the input difference of the 2nd round is determined by guessing 2 key bits according to Observation 3. Because we did not guess the input value of the inactive S-box in the first round, the input value of the key variable in the second round is quadratic, i.e. $\delta_K(\text{Enc}_K^{(1)}(X)) = 2$. According to

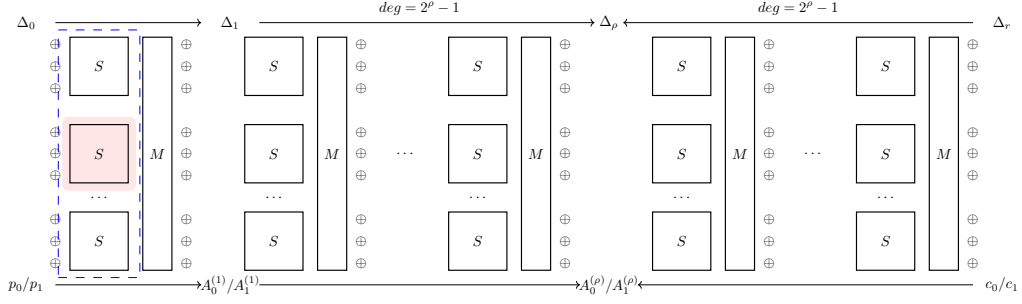


Figure 2: The attack setup for odd rounds for LowMC instances. The S-box shown in red is linearized when we activate only one bit. The S-box shown in blue is linearized when we activate one bit in each active S-box.

Appendix B, we know that

$$\begin{aligned} \Delta Enc_K^{(r)}[i] &= \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t}(\Delta Enc_K^{(r-1)}[t](Enc_K^{(r-1)}(X)[j] \oplus L_{r+1}(K)[j]) \\ &\quad \oplus \sum_{j=0}^{n-1} B_j(\Delta Enc_K^{(r-1)}[j]) \oplus \Delta Enc_K^{(r-1)}[j] \Delta Enc_K^{(r-1)}[t] \\ &\quad \oplus \Delta Enc_K^{(r-1)}[j](Enc_K^{(r-1)}(X)[t] \oplus L_{r+1}(K)[t]). \end{aligned}$$

Consequently,

$$\delta_K(\Delta Enc_K^{(r)}) = \max(\delta_K(\Delta Enc_K^{(r-1)}) + \max(\delta_K(Enc_K^{(r-1)}(X)), 1), 2\delta_K(\Delta Enc_K^{(r-1)})). \quad (4)$$

Let $r = 2$, it is easy to know that $\delta_K(\Delta Enc_K^{(2)}) = 0 + 2 = 2$. Recursively, we have

$$\delta_K(\Delta Enc_K^{(r)}) = 0 + 2 + 4 + \dots + 2^{r-1} = 2^r - 2.$$

The observation can be summarized as follows.

Observation 4. Let $Enc_K^{(R)}: \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an R -round LowMC block cipher defined above. Select an input difference Δ_0 such that it has only one active bit. Given two plaintexts X and X' such that $X \oplus X' = \Delta_0$, we have $\delta_K(\Delta Enc_K^{(r)}) = 2^r - 2$ after guessing two key bits which correspond to the 2 inactive bits of the active S-box.

Then we can obtain n equations of degree $2^{\rho+1} - 2$ in $n - 2$ variables for an odd number of rounds $r = 2\rho + 1$. This system of equations can be solved easily by Dinur's algorithm.

Complexity Evaluation. We use Dinur's algorithm to solve the equation system we obtained. Based on Equation (2) and choosing $N = 4$, the time complexity can be evaluated in bit operations as

$$T_{CP_1} = 4 \cdot T_{[Din21]}(n - 2, n_1, 2^{\rho+1} - 2, 4).$$

The memory complexity can be evaluated in bits as

$$M_{CP_1} = 8 \cdot (n_1 + 1) \cdot \binom{n-2-n_1}{\leq (2^{\rho+1}-2) \cdot (n_1+1) - n_1+1}.$$

For $n = 255$, $R = 5$, we have $\mathbf{d} = 2^{\rho+1} - 2 = 6$. If we choose $n_1 = 16$ and $\ell = n_1 + 1 = 17$, we get $T_{CP_1} \approx 2^{253.0}$ bit operations and $M_{CP_1} \approx 2^{228.9}$ bits.

Activate One Bit in Each Active S-box. Select an input difference such that the input difference of each active S-box has only one active bit. For active S-boxes, we guess two key bits which correspond to the 2 inactive bits of the S-box. For inactive S-boxes, we just guess two random input bits. Then, we obtain $\delta_K(Enc_K^{(1)}(X)) = 1$ and $\delta_K(\Delta Enc_K^{(1)}) = 0$. According to Equation (4) and let $r = 2$, we have

$$\delta_K(\Delta Enc_K^{(2)}) = \max(\delta_K(\Delta Enc_K^{(1)}) + \max(\delta_K(Enc_K^{(1)}(X)), 1), 2\delta_K(\Delta Enc_K^{(1)})).$$

It is easy to know that $\delta_K(\Delta Enc_K^{(2)}) = 0 + 1 = 1$. Recursively, we have

$$\delta_K(\Delta Enc_K^{(r)}) = 0 + 1 + 2 + \dots + 2^{r-2} = 2^{r-1} - 1.$$

The observation can be summarized as follows.

Observation 5. Let $Enc_K^{(R)}: \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be an R -round LowMC block cipher defined above. Select an input difference Δ_0 such that it has only one active bit for each active S-box. Given two plaintexts X and X' such that $X \oplus X' = \Delta_0$, we guess:

- two key bits corresponding to the inactive bits of the active S-box;
- two random key bits of the inactive S-box.

Then, we have $\delta_K(\Delta Enc_K^{(r)}) = 2^{r-1} - 1$.

Then we can construct the equation system as depicted in Figure 2.

Complexity Evaluation. To better solve this equation system, we use the two polynomial methods introduced in Section 2.3 to achieve a better time-memory trade-off.

Key Recovery Using Dinur's Algorithm. When $r = 2\rho + 1$, it is sufficient to guess $2n/3$ key variables to linearize the first round and determine the input difference of the second round. Then we can get n equations of degree $2^\rho - 1$ about $n/3$ variables. Based on Equation (2) and choosing $N = 4$, the time complexity can be evaluated in bit operations as

$$T_{CP_2^{(1)}} = 2^{2n/3} \cdot T_{[Din21]}(n/3, n_1, 2^\rho - 1, 4).$$

The memory complexity can be evaluated in bits as

$$M_{CP_2^{(1)}} = 8 \cdot (n_1 + 1) \cdot \binom{n/3 - n_1}{\leq (2^\rho - 1) \cdot (n_1 + 1) - n_1 + 1}.$$

For $n = 255$, $R = 7$, we have $d = 2^\rho - 1 = 7$. If we choose $n_1 = 4$ and $\ell = n_1 + 1 = 5$, we get $T_{CP_2^{(1)}} \approx 2^{261.8}$ bit operations and $M_{CP_2^{(1)}} \approx 2^{81.6}$ bits.

Key Recovery Using [SCW23]. Using the polynomial method in [SCW23], we can further optimize time complexity and memory complexity for some instances. After guessing the $2n/3$ key variables, the whole key space is effectively divided into several partial spaces. Suppose that the input of the i -th S-box is $(x_{3i} + k_{3i}, x_{3i+1} + k_{3i+1}, x_{3i+2} + k_{3i+2})$ and the input difference of the i -th S-box is $(\Delta x_{3i}, \Delta x_{3i+1}, \Delta x_{3i+2}) = (0, 0, 1)$. If we guess two key variables as $(k_{3i}, k_{3i+1}) = (g_{2i}, g_{2i+1})$, then the 3-bit key variable takes value from the set

$$B^{g_{2i}g_{2i+1}} = \{g_{2i}, g_{2i+1}, x_{3i} \oplus g_{2i} \oplus x_{3i+1} \oplus g_{2i+1} \oplus x_{3i+2} \\ \oplus y_i \oplus (x_{3i} \oplus g_{2i})(x_{3i+1} \oplus g_{2i+1}) \mid y_i \in \{0, 1\}\}.$$

The output difference can be computed as

$$(\Delta z_{3i}, \Delta z_{3i+1}, \Delta z_{3i+2}) = (x_{3i+1} + g_{2i+1}, x_{3i} + g_{2i}, 1),$$

where x_{3i} and x_{3i+1} are known.

So, after guessing $2m$ bits $G = (g_0, g_1, \dots, g_{2m-1}) \in \{0, 1\}^{2m}$, we obtain n equations of degree $d = 2^\rho - 1$ for $2\rho + 1$ round LowMC instance over the partial space. Based on Equation (3), the time complexity can be estimated as

$$T_{\text{CP}_2^{(2)}} = T_{[\text{SCW23}]}(N_u, n_1, D)$$

bit operations. The memory complexity can be estimated as

$$M_{\text{CP}_2^{(2)}} = \binom{N_u}{\leq D} \cdot n \cdot 2^{n_1/3} + 4 \cdot (n_1/3 + 1) \cdot 2^{N_u}$$

bits, where $N_u = (n - n_1)/3$ and $D = (2^\rho - 1) \cdot (n_1/3 + 1) - n_1/3$ bits. For $n = 255$, $R = 7$, we have $d' = 2^\rho - 1 = 7$. If we choose $n_1 = 3$ and $l = n_1/3 + 1 = 2$, we get $T_{\text{CP}_2^{(2)}} \approx 2^{260.7}$ bit operations and $M_{\text{CP}_2^{(2)}} \approx 2^{87.0}$ bits.

All parameter selections in Section 3 are provided in the Appendix A.

Comparison with Previous Works. Different from the analysis proposed in [BBVY21, SCW23], our work uses the derivation of the two plaintext/ciphertext pairs to purely reduce the algebraic degree of the equation system. In [Din21], the author also proposes an interesting property of the intermediate variables in LowMC to reduce the algebraic degree. Consider the 3 output polynomial of degree $2^{\rho+1}$ for an S-box, if we multiply these polynomials together, the degree is $4 \cdot 2^\rho$ instead of $3 \cdot 2^{\rho+1}$. Using this property, they successfully reduce the algebraic degree of 3-multiplied polynomials from 24 to 16 for 5-round LowMC and from 48 to 32 for 7-round LowMC. In our strategy, when activating only one bit in the input difference, the algebraic degree of the 3-multiplied polynomial reaches 18 for 5-round LowMC and 42 for 7-round LowMC, which is larger than that of [Din21]. So, even though our attack improves upon the results of [BBVY21, SCW23], it does not achieve better results compared with [Din21]. If we activate one bit in each active S-box, the algebraic degree of 3-multiplied polynomial is 9 for 5-round LowMC and 21 for 7-round LowMC, which is much smaller than that of [Din21]. However, our attack does not achieve better results for 5-round LowMC compared to [Din21], even under chosen-plaintext setting, due to the high cost of linearization. However, as the number of rounds increases, the algebraic degree plays a more significant role in the overall complexity, which provides a better trade-off. For $r = 7$, our strategy achieves better time complexity.

To make a more direct comparison, we provide detailed parameters for different attack methods when $r = 2\rho + 1$ is odd in Table 4. e and u denote the number of guess bits (linearization) and the number of variables, respectively. deg denotes the algebraic degree of polynomial and deg' denotes the algebraic degree of the 3-multiplied polynomial as mentioned above.

Table 2: Choices of e , u , deg and deg' in different work for odd-round attack.

e	u	deg	deg'	Ref.
2	$n - 2$	$2^{\rho+1} - 2$	$3 \cdot (2^{\rho+1} - 2)$	Sec. 3.2*
$2n/3$	$n/3$	$2^\rho - 1$	$3 \cdot (2^\rho - 1)$	Sec. 3.2**
-	n	$2^{\rho+1} - 1$	$3 \cdot (2^{\rho+1} - 1)$	Sec. 3.1
$n/3$	$2n/3$	2^ρ	$3 \cdot 2^\rho$	[BBVY21]
$2n/3$	$n/3$	2^ρ	$3 \cdot 2^\rho$	[SCW23]
-	n	$2^{\rho+1}$	$4 \cdot 2^\rho$	[Din21]

* Activate one bit in the input difference.

** Activate one bit in each active S-box.

4 Advanced Difference Enumeration Attack Framework for the Full S-Box Layer

In this section, we revisit the difference enumeration attack to explore the potential for further optimization of 4-round attacks. In [LIM21], the problems of enumerating compact differential trails and retrieving the full key for the full S-box layer are reduced to the problem of solving linear equation systems using guess-and-determine strategy. However, these enumerated variables denote the difference or intermediate states and there are some inherent relation inside them. We use T_d and T_k to represent the time complexity of the difference enumeration phase and key-recovery phase, respectively. N_d represents the number of potentially correct compact differential trails after the difference enumeration phase.

4.1 Difference Enumeration Phase

As shown in [LIM21], when enumerating the compact differential trails of 4-round LowMC, we construct linear equation system of Δ_2^S and Δ_3^S for each possible transition in the first two rounds. By choosing one Δ_0^S , we can optimize the number of possible differences in the 2nd round. However, the other 3 possible Δ_0^S is still out of control. So we need to enumerate around $4^{m-q} + 3 \times 4^m$ transitions in total and q denotes number of inactive S-boxes in the 2nd round. One natural way to reduce the cost is to exploit cubic equations instead of enumerating directly according to the following observation.

Observation 6. [LSW⁺22] Denote the input difference and output difference of the 3-bit LowMC S-box by $(\Delta x_0, \Delta x_1, \Delta x_2)$ and $(\Delta z_0, \Delta z_1, \Delta z_2)$, respectively. The following 2 cubic equations are sufficient to describe its DDT:

$$(1 \oplus \Delta x_0)(1 \oplus \Delta x_1)(1 \oplus \Delta x_2) = (1 \oplus \Delta z_0)(1 \oplus \Delta z_1)(1 \oplus \Delta z_2), \quad (5)$$

$$(1 \oplus \Delta x_0)(1 \oplus \Delta x_1)(1 \oplus \Delta x_2) = \Delta x_0 \Delta z_0 \oplus \Delta x_1 \Delta z_1 \oplus \Delta x_2 \Delta z_2 \oplus 1. \quad (6)$$

For 4-round LowMC, we can introduce $3m$ variables $(u_0, \dots, u_{3m-1})$ and $3m$ variables $(v_0, \dots, v_{3m-1})$ to represent the output difference of the S-boxes for the 2nd and 3rd rounds, respectively. Then $2m$ cubic equations of these $6m$ variables can be constructed according to Observation 6. For the 1st/4th round, we enumerate the possible difference forwards/backwards to Δ_1/Δ_3^S . For each S-box in the 2nd and 4th rounds, we can obtain at least one linear equation according to Observation 1. Thus, we can get at least m linear equations for $(u_0, \dots, u_{3m-1})$ and m linear equations for $(v_0, \dots, v_{3m-1})$. After Gaussian elimination, the number of variables in the cubic system can be reduced from $6m$ to $4m$. We then use Dinur's algorithm to solve this cubic equation system. However, the complexity of Dinur's algorithm grows rapidly as the number of variables increases. When $(n, k, m) = (129, 129, 43)$, we obtain 129 cubic equations for 172 variables after Gaussian elimination. According to Equation (2), the time complexity is $2^{164.2}$ bit operations and the memory complexity is $2^{140.2}$ bits when $n_1 = 22$. This time complexity is much greater than the complexity of Gray-code based exhaustive search technique in [BCC⁺10].

As mentioned above, it is necessary to develop a new technique to construct the nonlinear equation system for the difference enumeration phase. Instead of directly using Dinur's algorithm to solve the equation system, we split the whole equation system into 2 parts. For each part, we make use of the inherent relation in the variables to further reduce the complexity. An overview of the new attack framework is depicted in Figure 3.

For convenience, we denote by $a^{(i-1)}$ and $b^{(i-1)}$ the number of active S-boxes and inactive S-boxes in the i -th round respectively, where $1 \leq i \leq 4$. Obviously, we have $a^{(0)} = 1$ and $b^{(0)} = n/3 - 1$. We set lower bounds $b_{min}^{(1)}$ and $b_{min}^{(3)}$ for the number of inactive

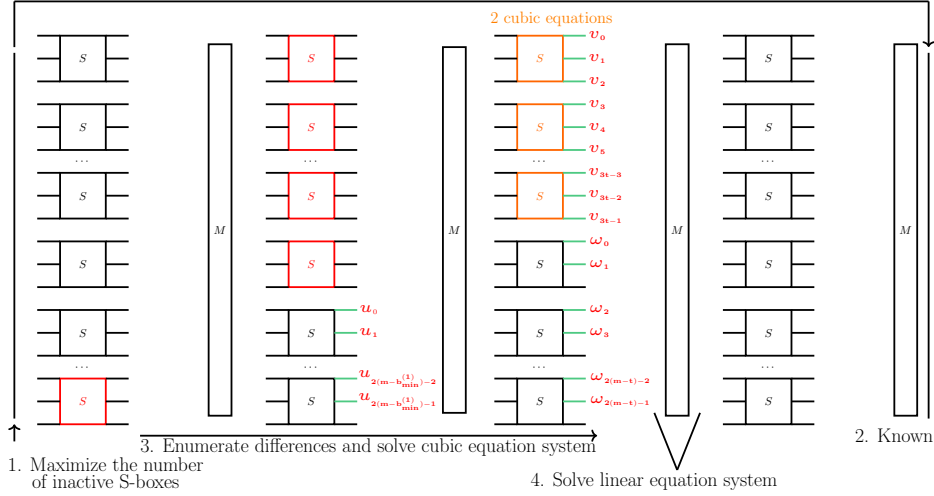


Figure 3: The improved difference enumeration attack framework for the full S-box layer. The S-boxes used to enumerate differences are colored in red. The S-boxes used to build cubic equation systems are colored in orange. The green line indicates where the variable is introduced.

S-boxes in the 2nd and 4th rounds, respectively. We only care about the difference trails that satisfy $b^{(1)} \geq b_{min}^{(1)}$ and $b^{(3)} \geq b_{min}^{(3)}$. The success probability will be discussed later.

We still enumerate the 4 possible differences in the first round. In the second round, we introduce $2(m - b_{min}^{(1)})$ variables $(u_0, \dots, u_{2(m-b_{min}^{(1)})-1})$ to represent Δ_1^S according to Observation 1. For the first t S-boxes in the 3rd round, we can introduce $3t$ variables $(v_0, \dots, v_{3t-1})$ to represent $\Delta_2^S[0: 3t-1]$. Based on Observation 6, we can obtain the corresponding $2t$ cubic equations. Notice that these $2(m - b_{min}^{(1)})$ variables $(u_0, \dots, u_{2(m-b_{min}^{(1)})-1})$ represent the output difference of $(m - b_{min}^{(1)})$ S-boxes, so they take value from a constrained space. We enumerate the $2^{1.86e}$ output differences⁵ of the first e S-boxes. For each possible transition, we can construct $2t$ cubic equations of $2(m - e - b_{min}^{(1)}) + 3t$ variables, which can be solved efficiently using Dinur's algorithm [Din21]. For each solution, the correctness can be checked via DDT.

Now only the output differences of the last $(m - t)$ S-boxes in the 3rd round are not determined. Since the input difference is known, it is sufficient to introduce $2(m - t)$ variables $(w_0, \dots, w_{2(m-t)-1})$ to represent the output difference. Based on Observation 1, we can construct at least $m - b_{min}^{(3)} + 3b_{min}^{(3)}$ linear equations of $2(m - t)$ variables. If $2(m - t) \leq m + 2b_{min}^{(3)}$, i.e. $t \geq m/2 - b_{min}^{(3)}$, we can easily solve them by Gaussian elimination. If $t < m/2 - b_{min}^{(3)}$, it is necessary to enumerate all the $2^{m-2t-2b_{min}^{(3)}}$ variables.

The whole procedure can be described as follows.

- Step 1. Choose an input difference Δ_0 such that there will be only one active S-box in the first round, i.e. $a^{(0)} = 1$ and $b^{(0)} = n/3 - 1$.
- Step 2. Encrypt a plaintext pair whose XOR difference is Δ_0 for r rounds and obtain the corresponding XOR difference Δ_r of the ciphertexts.
- Step 3. Enumerate 4 possible state differences in the first round. For each $\Delta_0^S[0: 3m-1]$:
 - 3.1 Introduce $2(m - b_{min}^{(1)})$ variables $(u_0, \dots, u_{2(m-b_{min}^{(1)})-1})$ to represent Δ_1^S and

⁵The average number of reachable output differences over the S-box for a uniformly randomly chosen input difference is around $2^{1.86}$ [RST18].

introduce $3t$ variables $(v_0, \dots, v_{3t-1})$ to represent $\Delta_2^S[0: 3t-1]$.

3.2 Enumerate $2^{1.86e}$ output differences for the first $2e$ variables in $(u_0, \dots, u_{2(m-b_{min}^{(1)})-1})$. For each possible transition, construct $2t$ cubic equations of $2(m-e-b_{min}^{(1)})+3t$ variables. Then we perform Dinur's algorithm on them and find the solution. The correctness can be checked via DDT. For each possible $\Delta_1^S[0: 3m-1]|\Delta_2^S[0: 3t-1]$, move to Step 4.

Step 4. For each possible $\Delta_1^S[0: 3m-1]|\Delta_2^S[0: 3t-1]$, we introduce $2(m-t)$ variables $(w_0, \dots, w_{2(m-t)-1})$ to represent $\Delta_2^S[3t: 3m-1]$. Compute the state difference backwards from Δ_r to Δ_3^S . Based on Observation 1, we construct at least $m+2b_{min}^{(3)}$ linear equations. If $2(m-t) \leq m+2b_{min}^{(3)}$, i.e. $t \geq m/2 - b_{min}^{(3)}$, we can solve them by Gaussian elimination. Otherwise, we need to enumerate $(m-2t-2b_{min}^{(3)})$ variables. The correctness can be checked via DDT.

Complexity Evaluation. In Step 3, according to Equation (2) and choosing $N = 16$, the time complexity can be evaluated in bit operations as

$$T_1 = 4 \cdot 2^{1.86e} \cdot T_{[\text{Din21}]}(2m - 2b_{min}^{(1)} - 2e + 3t, uu_1, 3, 16). \quad (7)$$

We choose the parameters (e, t, uu_1) such that

$$\begin{aligned} 0 < e < m - b_{min}^{(1)}, \quad 0 < t < m, \\ 2m - 2b_{min}^{(1)} - 2e + 3t > 0, \quad 0 < uu_1 < 2m - 2b_{min}^{(1)} - 2e + 3t, \\ 2t \geq 2 \cdot (uu_1 + 1) + 2, \quad 2m - 2b_{min}^{(1)} - 2e + 3t - uu_1 \geq 3 \cdot (uu_1 + 1) - uu_1 + 1. \end{aligned}$$

We use N_1 to represent the solution of the cubic equation system in Step 3.2. Note that Equation (5) and (6) hold with probabilities of $\frac{25}{32}$ and $\frac{9}{16}$, respectively. So the probability of them both being true is $\frac{225}{512}$. Then we can estimate N_1 as

$$N_1 = 2^{2m-2b_{min}^{(1)}-2e+3t} \cdot \left(\frac{225}{512}\right)^t.$$

In Step 4, if $t \geq m/2 - b_{min}^{(3)}$, there is at most one solution by using Gaussian elimination. The time complexity in this step is $T_2 = (m + 2b_{min}^{(3)})^3$. Otherwise, the time complexity in this step is $T_2 = 2^{m-2b_{min}^{(3)}-2t} \cdot (m + 2b_{min}^{(3)})^3$. So the total time complexity of the difference enumeration phase is $T_d = T_1 + 4 \cdot 2^{1.86e} \cdot N_1 \cdot T_2$ bit operations.

If using Dinur's memory-efficient Möbius transform, the memory complexity is evaluated in bits as

$$M = 8(uu_1 + 1) \left(\begin{array}{l} 2m - 2b_{min}^{(1)} - 2e + 3t - uu_1 \\ \leq 3 \cdot (uu_1 + 1) - uu_1 + 1 \end{array} \right) + (m + 2b_{min}^{(1)})^2.$$

The time complexity is affected by both $(b_{min}^{(1)}, b_{min}^{(3)})$ and the choice of (e, t, uu_1) , which collectively define the theoretical upper bound on time complexity. In Table 3, we select the best (e, t, uu_1) and $(b_{min}^{(1)}, b_{min}^{(3)})$ for different instances.

Probability of An Isolated Solution. Since we use Dinur's algorithm to solve the equation system in Step 3, it is necessary to demonstrate that the system admits an isolated solution with high probability. The variable partition associates a solution to the equation system with $2^{uu_1} - 1$ different assignments, where each possible assignment satisfies the system of $2t$ equations with probability $(\frac{225}{512})^t$. Therefore, the probability that the system admits an isolated solution is at least $1 - 2^{uu_1} \cdot (\frac{225}{512})^t$, which is typically very close to 1 according to Table 3. A more detailed analysis is on page 14 of reference [Din21].

4.2 Key-Recovery Phase

In the key-recovery phase, we show how to retrieve the full key from the recovered compact differential trails. Due to the influence of the full S-box layer, the number of compact differential trails increases rapidly, $N_d = 2^{1.86m \cdot r - n}$. To keep $N_d T_k < 2^k$, it is very important to further optimize T_k . Instead of the simple guess-and-determine strategy, we utilize more information of the S-boxes to construct the linear equation system. Set two counters $\mathbf{a}$ and $\mathbf{b}$. We only consider the active S-boxes of the last three rounds and the inactive S-boxes of the last two rounds. For each differential trail, start from the ciphertext and check the S-boxes of the last three rounds in the backward direction round by round and one by one.

1. If the S-box is active, increase $\mathbf{a}$ by 1. According to Observation 2, we can obtain 2 linear equations in terms of the key and the intermediate variables.
2. If the S-box is inactive of the last two rounds, increase $\mathbf{b}$ by 1 and introduce 3 intermediate variables to represent its input. Otherwise, $\mathbf{b}$ remains unchanged and no variables are introduced.

If $2\mathbf{a} \geq 3\mathbf{b} + n$, we only need to solve $2\mathbf{a}$ linear equations to uniquely determine the key. Otherwise, applying Gaussian elimination on the first $2\mathbf{a}$ linear equations will allow us to obtain $3\mathbf{b} + n - 2\mathbf{a}$ free variables. We only need to enumerate them. The correctness of the solution can be easily verified via a single plaintext/ciphertext pair.

Choice of the Threshold. Obviously, we have $\mathbf{a} = \sum_{i=1}^{i=3} a^{(i)}$ and $\mathbf{b} = b^{(2)} + b^{(3)}$. Since the time complexity of the key-recovery phase is affected by the number of inactive S-boxes $\mathbf{b}$, we choose the threshold $b_{max}^{(i)}$ for each of the last three rounds such that the sum of the number of inactive S-boxes in the last three rounds reaches $\sum_{i=1}^{i=3} b_{max}^{(i)}$ is the worst case, i.e. when the sum of the number of inactive S-boxes in the last three rounds is less than $\sum_{i=1}^{i=3} b_{max}^{(i)}$, the time complexity will not be higher. At this time, the minimum number of active S-boxes for each of the last three rounds is $a_{min}^{(i)} = m - b_{max}^{(i)}$. In other words, the time complexity is highest when $\mathbf{a} = \sum_{i=1}^{i=3} a_{min}^{(i)}$ and $\mathbf{b} = b_{max}^{(2)} + b_{max}^{(3)}$.

To ensure that the success probability is not affected too much, we choose the threshold $b_{max}^{(i)} = \lfloor 0.26m \rfloor$ based on the well-known statistical property

$$\Pr(b^{(i)} \leq b_{max}^{(i)}) = \sum_{i=0}^{\lfloor 0.26m \rfloor} \binom{m}{i} \times \left(\frac{7}{8}\right)^{m-i} \times \left(\frac{1}{8}\right)^i \approx 1,$$

where $1 \leq i \leq 3$. Considering that $b_{min}^{(1)}$ and $b_{min}^{(3)}$ are set in the difference enumeration phase, the success probability can be calculated as

$$Pr = \sum_{\substack{i=3 \\ k=\sum_{i=1}^{i=3} a_{min}^{(i)}}}^{3m-b_{min}^{(1)}-b_{min}^{(3)}} \sum_{j=a_{min}^{(2)}}^m \sum_{i=a_{min}^{(1)}}^{m-b_{min}^{(1)}} \binom{m}{i} \cdot \binom{m}{j} \cdot \binom{m}{k-j-i} \cdot \left(\frac{7}{8}\right)^k \cdot \left(\frac{1}{8}\right)^{3m-k},$$

where $a_{min}^{(3)} \leq k - j - i \leq m - b_{min}^{(3)}$.

Since $b_{max}^{(1)} = b_{max}^{(2)} = b_{max}^{(3)}$, for convenience, we represent $b_{max}^{(i)}$ with b_{max} and $1 \leq i \leq 3$. In Table 3, we select the best (e, t, uu_1) so that the time complexity is optimal. The values of $(b_{min}^{(1)}, b_{min}^{(3)})$ and b_{max} in different cases are also given.

Table 3: Choices of $(e, t, uu_1, b_{min}^{(1)}, b_{min}^{(3)}, b_{max})$ for different (n, k, m)

r	n	k	m	e	t	uu_1	$b_{min}^{(1)}$	$b_{min}^{(3)}$	b_{max}	Pro.
4	129	129	43	37	11	9	4	4	11	0.63
4	192	192	64	58	19	7	5	5	16	0.84
4	192	192	64	60	19	17	3	3	16	0.97
4	255	255	85	75	26	7	7	7	22	0.84
4	255	255	85	78	26	7	4	4	22	0.99

Complexity Evaluation of Key-Recovery Phase. As mentioned above, the time complexity is the worst case when $\mathbf{a} = \sum_{i=1}^3 a_{min}^{(i)}$ and $\mathbf{b} = b_{max}^{(2)} + b_{max}^{(3)}$. If $2 \cdot \mathbf{a} \geq 3 \cdot \mathbf{b} + n$, we only need to solve $2 \cdot \mathbf{a}$ linear equations and $T_k = (2 \cdot \mathbf{a})^3$. Otherwise, there will be $3 \cdot \mathbf{b} + n - 2 \cdot \mathbf{a}$ free variables and $T_k = 2^{3 \cdot \mathbf{b} + n - 2 \cdot \mathbf{a}} \cdot (2 \cdot \mathbf{a})^3$. Considering $b_{min}^{(1)}$ and $b_{min}^{(3)}$, the estimated number of differential trails that can be recovered is

$$N_d = 4 \cdot 2^{1.86 \cdot (3 \cdot m - b_{min}^{(1)} - b_{min}^{(3)}) - n}.$$

Therefore, the overall complexity of the key recovery phase can be estimated as $N_d T_k$ as the time complexity of verifying candidate solutions is negligibly.

Attack on {192,192,64,4}. For $(n, k, m, R) = (192, 192, 64, 4)$, we choose $(e, t, uu_1, b_{min}^{(1)}, b_{min}^{(3)}, b_{max}) = (60, 19, 17, 3, 3, 16)$. The success probability $Pr \approx 0.97$. As discussed in Section 4.1, T_1 can be evaluated in bit operations as

$$T_1 = 4 \cdot 2^{1.86 \times 60} \cdot T_{[\text{Din21}]}(2 \times 64 - 2 \times 3 - 2 \times 60 + 3 \times 19, 17, 3, 16) \approx 2^{181.74}$$

and $N_1 \approx 2^{36.46}$. The time complexity of Gaussian elimination of linear equations is $T_2 \approx 2^{38.39}$ bit operations. So, the time complexity of the difference enumeration phase can be evaluated in bit operations as

$$T_d = T_1 + 4 \cdot 2^{1.86 \times 60} \cdot N_1 \cdot T_2 \approx 2^{188.4}.$$

In the key-recovery phase, $2 \cdot \mathbf{a} \geq 3 \cdot \mathbf{b} + n$, which means that there will be at most one solution after Gaussian elimination. So, $T_k = (2 \cdot \mathbf{a})^3 \approx 2^{24.5}$ bit operations. The estimated number of differential trails that can be recovered is $N_d = 2^{155.96}$ and the total time complexity of the difference enumeration attack is $T = T_d + N_d \cdot T_k \approx 2^{188.4}$ bit operations. The memory complexity of the difference enumeration attack is $M \approx 2^{49.2}$ bits.

5 Experiments

To verify the correctness of our attacks in Section 3 and Section 4, we performed experiments on concrete LowMC instances. These LowMC instances are generated with the official reference code⁶. The source code of our experiments is available at <https://anonymous.4open.science/r/Improved-Attacks-on-LowMC-AE75>.

Verifying the Attack in Section 3. We use a smaller instance with $(n, k, m, R) = (15, 15, 3, 5)$ to construct the equation system. We take the whitening key as the master key and guess $(k_{3i}, k_{3i+1}) = (k_{3i}^*, k_{3i+1}^*)$ for $i \in \{0, 1, \dots, 4\}$. Choose an input difference denoted by Δ_0 , where only the last bit is inactive and randomly choose two plaintexts with difference Δ_0 . Then, we get 15 equations of degree 3 with 5 variables and solve them using the method described in Section 3.2. For the verification part, we verify the independence of the system of equations and it shows that the equations are independent.

⁶<https://github.com/LowMC/lowmc>

Verifying the Attack in Section 4. Firstly, we verify the success probability by performing experiments on the instance with $(n, k, m, R) = (129, 129, 43, 4)$. We randomly choose 800 plaintext pairs such that there is only one active S-box in the first round. For each plaintext pair, we record the corresponding R -round differential trail during encryption and count the number of inactive S-boxes in the last 3 rounds. N_p denotes the number of plaintext pairs such that the number of inactive S-boxes in the 2nd round and the 4th round lies between 4 and 11 and the number of inactive S-boxes in the 3rd round is at most 11. We observe that $N_p/800 \approx 0.63$, indicating that the success probability is correct. Secondly, we verify the difference enumeration phase and set $e = 35$, $t = 13$, $uu_1 = 6$. Based on Section 4, we enumerate differences in the first round and the 35 S-box in the second round. We then introduce variables $(u_{70}, u_{71}, \dots, u_{77})$ and $(v_0, v_1, \dots, v_{38})$ and construct a system of 76 cubic equations. This equation system can be solved by Dinur's algorithm while the last round only requires Gaussian elimination. Finally, we perform key recovery. We use a differential trail with only one active S-box in the first round and the numbers of inactive S-boxes for the 2nd, 3rd and 4th rounds are 6, 5 and 7 respectively. A total of 222 linear equations over 165 variables can be established and the unique solution can be obtained by Gaussian elimination.

6 Conclusion

In this work, aiming to advance the state-of-the-art key recovery attacks on LowMC instances with full S-box layers using two plaintext/ciphertext pairs, we developed a new algebraic key-derived attack framework and proposed an improved differential enumeration attack framework. As proof of concept, we obtain the first 8-round attacks on LowMC for the very first time, and identify first attacks on three LowMC instances that cannot be attacked using previous methods. Better attacks are further obtained on LowMC from 4 to 7 rounds for almost all three instances. All these results given in this paper, as well as our new attack frameworks, we believe, could provide more insights into the cryptanalysis of LowMC with low allowable data complexity, which has not yet come to an end.

References

- [ARS⁺15] Martin R Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for mpc and fhe. In *Advances in Cryptology–EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part I 34*, pages 430–454. Springer, 2015.
- [BBDV20] Subhadeep Banik, Khashayar Barooti, F Betul Durak, and Serge Vaudenay. Cryptanalysis of lowmc instances using single plaintext/ciphertext pair. *IACR Transactions on Symmetric Cryptology*, 2020(ARTICLE):130–146, 2020.
- [BBVY21] Subhadeep Banik, Khashayar Barooti, Serge Vaudenay, and Hailun Yan. New attacks on lowmc instances with a single plaintext/ciphertext pair. In *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part I 27*, pages 303–331. Springer, 2021.
- [BCC⁺10] Charles Bouillaguet, Hsieh-Chung Chen, Chen-Mou Cheng, Tung Chou, Ruben Niederhagen, Adi Shamir, and Bo-Yin Yang. Fast exhaustive search for polynomial systems in $\mathbb{F}_2$. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 203–218. Springer, 2010.

- [BCV24] Subhadeep Banik, Andrea Caforio, and Serge Vaudenay. New attacks on lowmc using partial sets in the single-data setting. *cic*, 1, Issue 1, 2024.
- [CDG⁺17] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 1825–1842. ACM, 2017.
- [DEM16] Christoph Dobraunig, Maria Eichlseder, and Florian Mendel. Higher-order cryptanalysis of lowmc. In *Information Security and Cryptology-ICISC 2015: 18th International Conference, Seoul, South Korea, November 25-27, 2015, Revised Selected Papers 18*, pages 87–101. Springer, 2016.
- [Din21] Itai Dinur. Cryptanalytic applications of the polynomial method for solving multivariate equation systems over $\text{gf}(2)$. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 374–403. Springer, 2021.
- [DLMW15] Itai Dinur, Yunwen Liu, Willi Meier, and Qingju Wang. Optimized interpolation attacks on lowmc. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 535–560. Springer, 2015.
- [LIM21] Fukang Liu, Takanori Isobe, and Willi Meier. Cryptanalysis of full lowmc and lowmc-m with algebraic techniques. In *Advances in Cryptology-CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part III 41*, pages 368–401. Springer, 2021.
- [LMSI22] Fukang Liu, Willi Meier, Santanu Sarkar, and Takanori Isobe. New low-memory algebraic attacks on lowmc in the picnic setting. *IACR Transactions on Symmetric Cryptology*, pages 102–122, 2022.
- [LSW⁺22] Fukang Liu, Santanu Sarkar, Gaoli Wang, Willi Meier, and Takanori Isobe. Algebraic meet-in-the-middle attack on lowmc. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 225–255. Springer, 2022.
- [pic18] The Picnic Signature Algorithm Specification. <https://github.com/microsoft/Picnic/blob/master/spec/spec-v2.0.pdf>, 2018.
- [pic20] The Picnic Signature Algorithm Specification. <https://github.com/microsoft/Picnic/blob/master/spec/spec-v3.0.pdf>, 2020.
- [RST18] Christian Rechberger, Hadi Soleimany, and Tyge Tiessen. Cryptanalysis of low-data instances of full lowmcv2. *IACR Transactions on Symmetric Cryptology*, pages 163–181, 2018.
- [SCW23] Yimeng Sun, Jiamin Cui, and Meiqin Wang. Improved attacks on lowmc with algebraic techniques. *IACR Trans. Symmetric Cryptol.*, 2023(4):143–165, 2023.

A Summary of Attacks on LowMC Instances with Full S-box Layers in Section 3

Table 4: Summary of attacks on LowMC instances with full S-box layers in Section 3. T and M represent the time and memory complexity (in $\log_2$), respectively. All of them have a success probability of 0.69.

r	n	k	m	T	M	u_1	ℓ/l	Ref.			
4	129	129	43	126.1	106.3	16	17	Sec. 3.1			
	192	192	64	182.1	154.5	24	25	Sec. 3.1			
	255	255	85	237.9	202.6	32	33	Sec. 3.1			
5	129	129	43	134.4	122.9	7	8	Sec. 3.1			
				133.4	118.9	8	9	Sec. 3.2*			
				133.3	45.0	3	2	Sec. 3.2**			
	192	192	64	194.9	180.7	11	12	Sec. 3.1			
				193.3	174.0	12	13	Sec. 3.2*			
				195.5	56.7	8	9	Sec. 3.2**			
	255	255	85	254.9	235.2	14	15	Sec. 3.1			
				253.0	228.8	16	17	Sec. 3.2*			
				256.5	73.4	11	12	Sec. 3.2**			
6	129	129	43	134.4	122.9	7	8	Sec. 3.1			
	192	192	64	194.9	180.7	11	12	Sec. 3.1			
	255	255	85	254.9	235.2	14	15	Sec. 3.1			
7	129	129	43	136.4	39.8	1	2	Sec. 3.2**			
				199.8	183.1	4	5	Sec. 3.1			
				199.4	63.3	3	4	Sec. 3.2**			
	192	192	64	197.7	66.0	3	2	Sec. 3.2**			
				261.7	246.1	6	7	Sec. 3.1			
				261.8	81.6	4	5	Sec. 3.2**			
8	255	255	85	260.7	87.0	3	2	Sec. 3.2**			
				192	192	64	199.8	183.1	4	5	Sec. 3.1
				255	255	85	261.7	246.1	6	7	Sec. 3.1

* Activate one bit in the input difference.

** Activate one bit in each active S-box.

B The Proof of Theorem 1

In this section, we present a detailed proof of Theorem 1.

Proof. When $r = 1$, considering the i -th output bit of $Enc_K^{(1)}(X)$, we have

$$Enc_K^{(1)}(X)[i] = A \oplus \sum_{j=0}^{n-1} B_j(x_j \oplus k_j) \oplus \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t}(x_j \oplus k_j)(x_t \oplus k_t),$$

And

$$\Delta Enc_K^{(1)}[i] = \sum_{j=0}^{n-1} B_j \Delta x_j \oplus \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t}(\Delta x_t(x_j \oplus k_j) \oplus \Delta x_j(x_t \oplus k_t) \oplus \Delta x_j \Delta x_t)$$

Obviously, $\Delta Enc_K^{(1)}[i]$ is a linear equation of K . So we have $\delta_K(\Delta Enc_K^{(1)}) = 2^1 - 1 = 1$.

We proceed by induction on r . Assuming that this theorem holds for $r = s$, we now prove that it also holds for $r = s + 1$. Similarly, considering the i -th output bit of $Enc_K^{(s+1)}(X)$ we can get

$$\begin{aligned} Enc_K^{(s+1)}(X)[i] &= A \oplus \sum_{j=0}^{n-1} B_j(Enc_K^{(s)}(X)[j] \oplus L_{s+1}(K)[j]) \\ &\oplus \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t}(Enc_K^{(s)}(X)[j] \oplus L_{s+1}(K)[j])(Enc_K^{(s)}(X)[t] \oplus L_{s+1}(K)[t]), \end{aligned}$$

and

$$\begin{aligned} \Delta Enc_K^{(s+1)}[i] &= \sum_{j=0}^{n-1} \sum_{t=j+1}^{n-1} C_{j,t}(\Delta Enc_K^{(s)}[t](Enc_K^{(s)}(X)[j] \oplus L_{s+1}(K)[j]) \\ &\oplus \sum_{j=0}^{n-1} B_j(\Delta Enc_K^{(s)}[j] \oplus \Delta Enc_K^{(s)}[j] \Delta Enc_K^{(s)}[t]) \\ &\oplus \Delta Enc_K^{(s)}[j](Enc_K^{(s)}(X)[t] \oplus L_{s+1}(K)[t])). \end{aligned} \quad (8)$$

Since $\delta_K(\Delta Enc_K^{(s)}[i]) = 2^s - 1$ and $\delta_K(Enc_K^{(s)}(X)[i]) = 2^s$, we have $\delta_K(\Delta Enc_K^{(s+1)}[i]) = 2^{s+1} - 1$. □

C Comparison under Low Success Probability

In this section, we compare the time complexity of our work with [LIM21] under a low success probability. The time complexity of our attack primarily depends on the parameters $(e, t, uu_1, b_{min}^{(1)}, b_{min}^{(3)}, b_{max})$. By properly choosing the parameters (as explained in Section 4), we can significantly reduce the time complexity while keeping the attack effective. A summary of our results is provided in Table 5.

Table 5: Summary of the attacks on LowMC under low success probability. T , M and Pro. represent the time, memory complexity (in $\log_2$) and success probability, respectively.

(n, k, m, r)	e	t	uu_1	$b_{min}^{(1)}$	$b_{min}^{(3)}$	b_{max}	T	M	Pro.	Ref.
$(129, 129, 43, 4)$	-	-	-	-	-	-	121.0	-	0.24	[LIM21]
	37	11	9	5	5	11	120.9	32.3	0.40	Ours
$(192, 192, 64, 4)$	-	-	-	-	-	-	174.2	-	0.247	[LIM21]
	54	13	11	8	8	16	170.9	38.6	0.307	Ours
$(255, 255, 85, 4)$	-	-	-	-	-	-	227.0	-	0.2465	[LIM21]
	73	22	7	10	10	22	222.8	57.9	0.3967	Ours